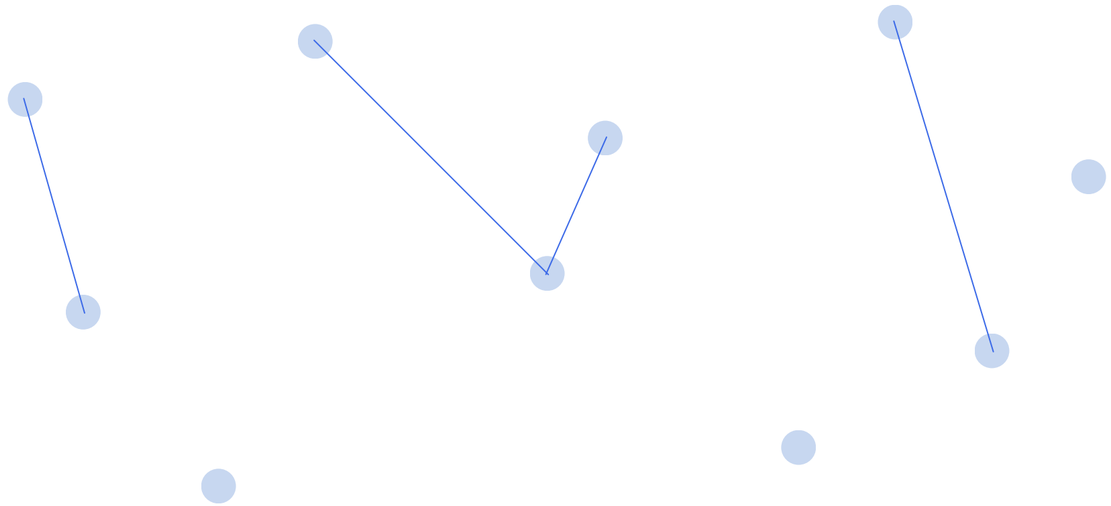
MES LAB PROJECT

Smart Environmental Monitoring & Alert System

STM32 Nucleo-F103RB · Embedded C · Python Dashboard

B.Tech Electronics & Communication Engineering

Academic Year 2025 – 2026



Group Members

Swapnil Saraswat (24EC043)

Amisha Singh (24EC103)

Ananya Kasaudhan (24EC104)

Pranjal Sharma (24EC111)

Microprocessor & Embedded Systems Lab

Problem Statement

The air we breathe is silently deteriorating — and most spaces have no system to warn us.

Toxic Gas Emissions

Industrial & vehicular pollution raises gas levels beyond safe thresholds undetected

Fire & Smoke Events

Fires go undetected for critical minutes, causing preventable loss of life & property

Temperature Hazards

Unmonitored temperature extremes damage equipment and endanger occupants silently

Undetected Intrusions

Motion in restricted zones goes unnoticed without an embedded detection system

Existing solutions are fragmented, expensive, cloud-dependent, or lack real-time alerting. A unified low-cost embedded system is the need of the hour.

Motivation & Background

7M+

Deaths/year from air pollution (WHO)

60%

Industrial accidents from undetected gas leaks

18.5%

Annual growth of smart sensor market

Why This Approach?

- STM32 Nucleo-F103RB offers a powerful, affordable platform with rich peripheral support
- Complete BOM under ₹2,000 — accessible for academic labs and student projects
- Live UART-to-Python pipeline creates a professional real-time data dashboard
- No internet required — fully standalone embedded system, zero cloud dependency
- Demonstrates full-stack embedded design: sensor → MCU → display → visualization

Objectives of the Project

01 Detect air quality, smoke, temperature & motion using multi-sensor integration on STM32 Nucleo-F103RB

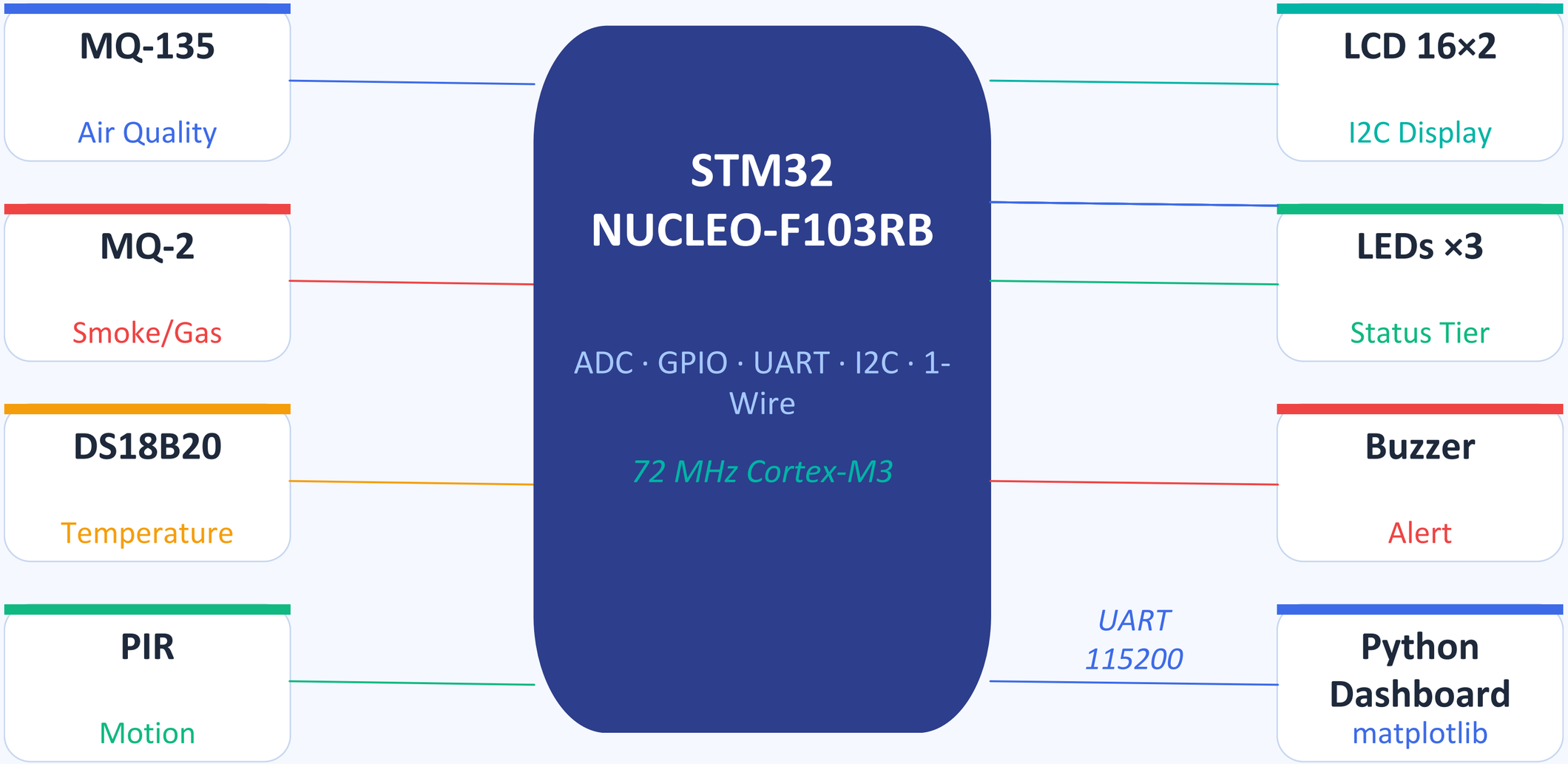
02 Trigger context-aware three-tier alerts via LED indicators and buzzer when thresholds are exceeded

03 Display real-time sensor readings on a 16×2 I2C LCD for on-site monitoring without external devices

04 Stream live sensor data over UART to a Python matplotlib dashboard for graphical real-time visualization

05 Deliver a complete, low-cost, lab-demonstrable embedded environmental monitoring system under ₹2,000

System Block Diagram



Part 1 · Member 1

Real-World Applications

Industrial Safety

Monitor toxic gases & fire risk in factories and chemical plants

Smart Homes

Automate ventilation & alert residents to smoke or intruders

Healthcare Facilities

Maintain sterile air quality and controlled temperature zones

Schools & Offices

Ensure healthy indoor air in crowded, enclosed spaces

Tunnels & Metro

Detect CO buildup and smoke in underground confined spaces

Greenhouses

Monitor temperature & gas levels for optimal crop growth

Social · Environmental · Economic Impact

Social Impact	Environmental	Economic
Early warning saves lives during fire emergencies	Enables data-driven environmental policy	Full BOM cost < ₹2,000 (vs ₹5k–50k commercial)
Reduces illness from chronic air pollution	Quantifies indoor pollution sources	Prevents losses from pollution-related illness
Accessible safety tech for low-income spaces	Promotes responsible gas usage monitoring	Preventive maintenance protects industrial assets
Raises awareness of indoor air quality	Reduces energy waste via smart auto-ventilation	Scalable to commercial product & startup

Comparison with Existing Solutions

Parameter	This Project (STM32)	Commercial IoT Device	Manual Monitoring
Cost	< ₹2,000	₹5,000 – ₹50,000	Low (no tech)
Real-Time Alert	LED + Buzzer	App notification	No
Multi-Sensor	4 sensors unified	Often single sensor	Single param
Live Dashboard	Python matplotlib	Cloud app	No
Internet Required	Standalone only	Wi-Fi/Cloud needed	No
Customizable	Fully open source	Limited/proprietary	None
Power	USB 5V	5–12V + Wi-Fi module	Human effort

Future Scope — Roadmap

Phase 1
Current
Multi-sensor STM32 system with live Python dashboard

Phase 2
Near-term
ESP8266 Wi-Fi module for wireless sensor streaming

Phase 3
Mid-term
Cloud (AWS IoT) with historical analytics & trends

Phase 4
Advanced
ML anomaly detection and predictive alert system

Phase 5
Product
Custom PCB design, enclosure & commercial deployment

Architecture designed for phased upgrades — no hardware redesign needed.

Market Relevance & Industry Alignment

$12.8B

Global Air Quality Monitoring Market by 2028

18.5%

CAGR Smart Sensor Market 2023–2028

3.2B+

IoT Devices in Environmental Sector

Industry Alignment

Industry 4.0	Smart factory environmental compliance and worker safety
Smart Buildings	ASHRAE, LEED, WHO indoor air quality standards alignment
IoT & Edge	Fits STM32 embedded-ML roadmap and edge-compute patterns
Academic	Full ADC, UART, I2C, GPIO usage in a real-world context

Hardware Overview — Component List

STM32 Nucleo-F103RB
Microcontroller

MQ-135 Sensor
Air Quality

MQ-2 Sensor
Smoke / Gas

DS18B20
Temperature

PIR Sensor
Motion Detection

LCD 16×2 I2C
Display

LEDs ×3
Status Indicator

Buzzer
Alert Output

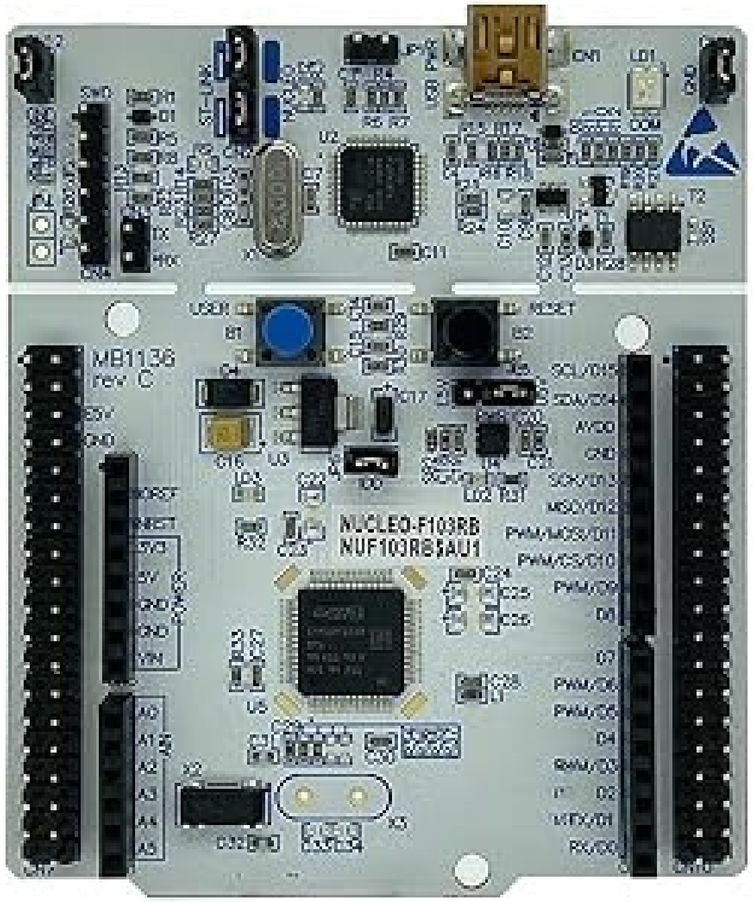
220Ω Resistors ×3
LED Protection

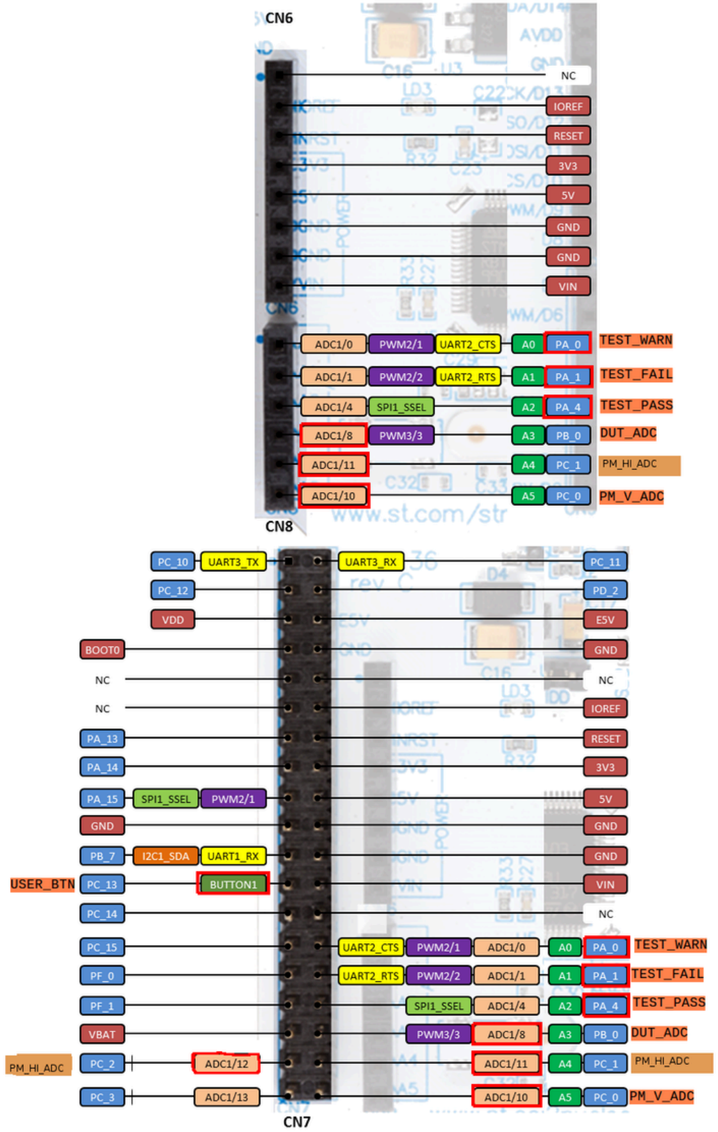
4.7kΩ Resistor
DS18B20 Pull-up

Breadboard
Prototyping

Jumper Wires
Connections

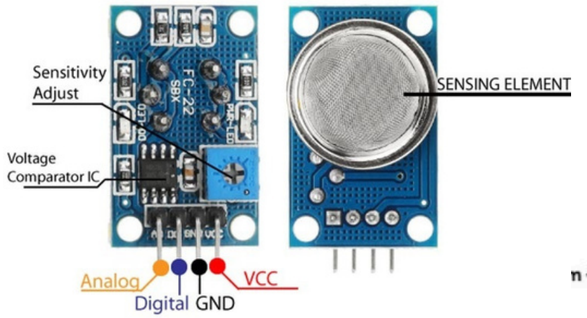
STM32F103RB- NUCLEO BOARD

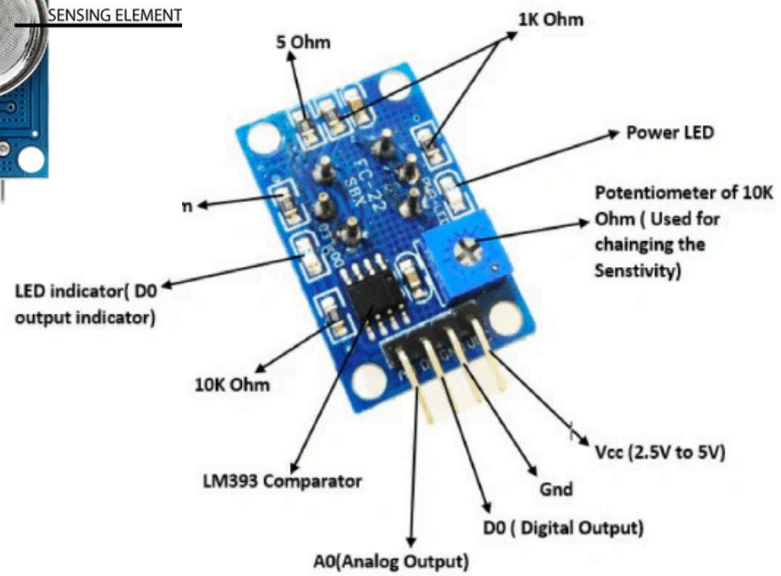




Part 3 · Member 3

MQ-135 — Air Quality Sensor





Working Principle

Uses a tin dioxide (SnO_2) sensing layer whose resistance decreases when exposed to target gases. Analog output voltage is proportional to gas concentration — higher ppm = higher voltage = higher ADC reading.

Specifications

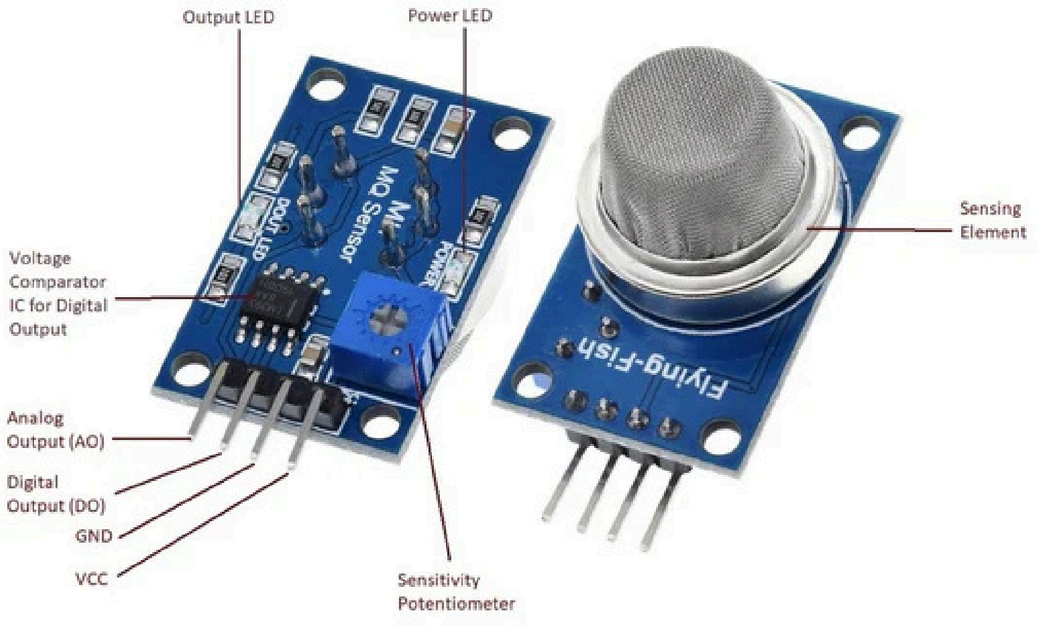
Parameter	Value
Operating Voltage	5V DC
Output Type	Analog (AOUT)
Detects	NH_3, CO_2, Benzene, Smoke
Preheat Time	~24 hours
STM32 Pin	PA0 — ADC1_IN0
ADC Resolution	12-bit (0–4095)

Role in Project

Reads raw 12-bit ADC value. Below 2000 = Good (Green LED). 2000–2500 = Poor (Yellow). Above 2500 = Danger (Red + Buzzer).

Part 3 · Member 3

MQ-2 — Smoke & Gas Sensor



Working Principle

Semiconductor sensor where SnO_2 conductance increases with gas concentration. Sensitive to LPG, propane, hydrogen, smoke and methane. Analog voltage output scales with detected gas level.

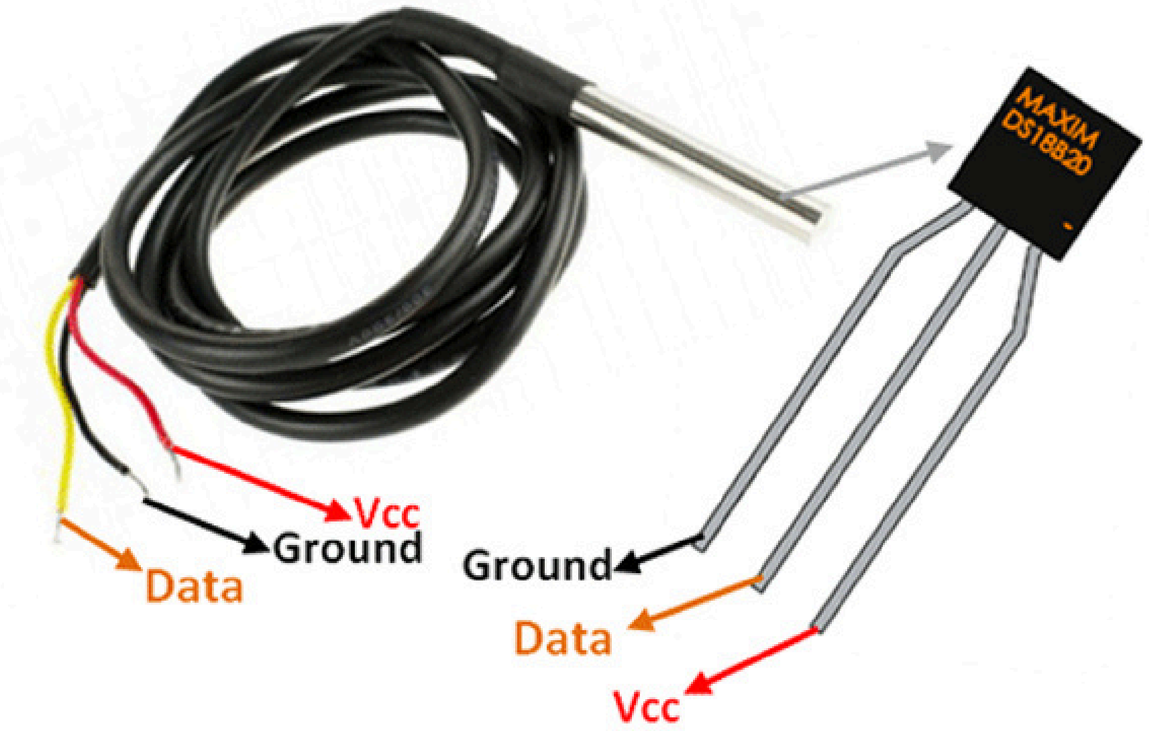
Role in Project

ADC value above 2500 triggers immediate Red LED and Buzzer alert. Smoke flag sent via UART to Python dashboard.

Specifications

Parameter	Value
Operating Voltage	5V DC
Output	Analog (AOUT)
Detects	LPG, Smoke, H_2, Propane
Sensitivity	300–10,000 ppm
Response Time	< 10 seconds
STM32 Pin	PA1 — ADC1_IN1

DS18B20 — Digital Temperature Sensor



Working Principle

1-Wire digital sensor using a silicon bandgap reference. Communicates via a single data line with a unique 64-bit ROM address. Internally converts temperature to 12-bit digital value without any external ADC.

Specifications

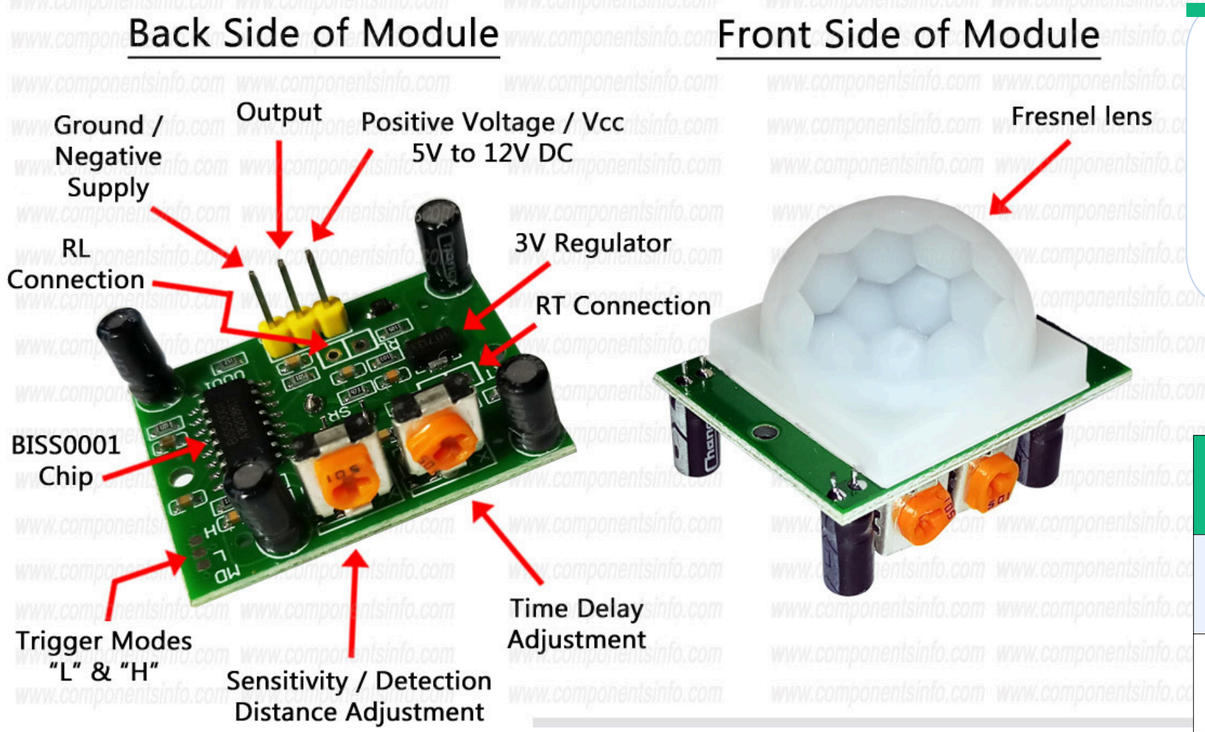
Parameter	Value
Protocol	1-Wire (Single Pin)
Range	−55°C to +125°C
Accuracy	±0.5°C (−10°C to +85°C)
Resolution	9–12 bit selectable
Supply Voltage	3.0V – 5.5V
STM32 Pin	PB9 (GPIO_OUT)

Role in Project

Polled every 1 second using custom 1-Wire driver in HAL. Needs 4.7kΩ pull-up to 3.3V. Temperature displayed on LCD line 1.

Part 3 · Member 3

PIR Sensor — Motion Detection



Working Principle

Passive Infrared sensor detects changes in infrared radiation from warm bodies. Two pyroelectric elements produce a differential signal when a person moves through the detection cone.

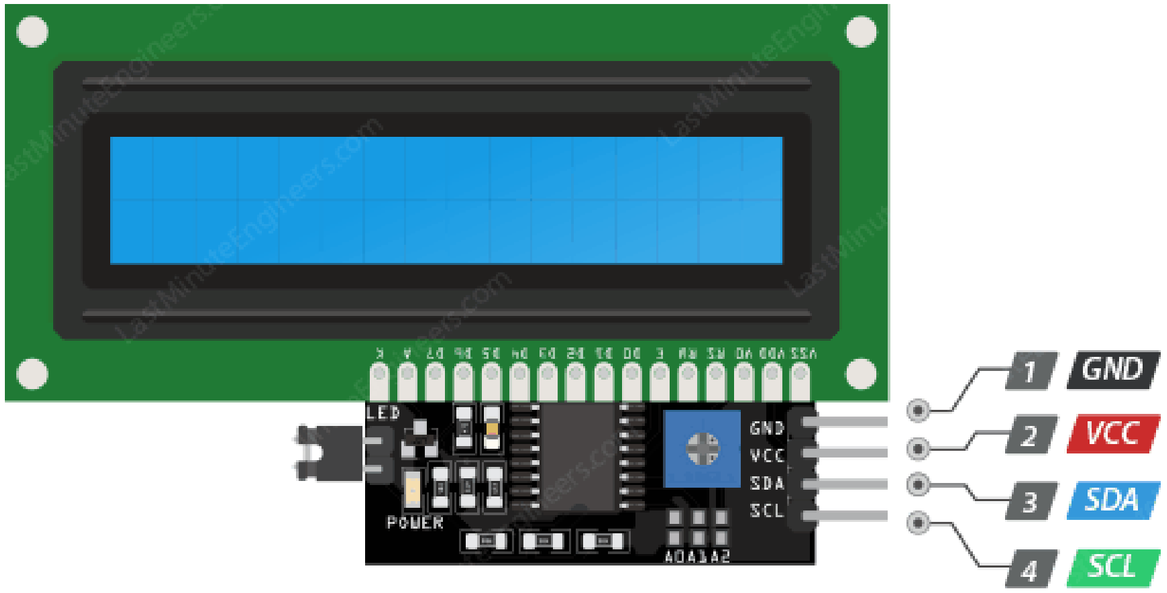
Specifications

Parameter	Value
Operating Voltage	5V – 20V
Detection Range	Up to 7 metres
Detection Angle	~110°
Output	Digital HIGH/LOW (3.3V/0V)
Trigger Mode	Adjustable: 0.5s – 5 minutes
STM32 Pin	PB10 — GPIO_INPUT

Role in Project

HIGH output detected via GPIO read in main polling loop. Motion flag sent over UART and shown on LCD line 2.

LCD 16×2 with I2C Module



Working Principle

HD44780-compatible 16×2 character LCD driven via PCF8574 I2C expander. Reduces wiring from 8+ data pins to just 2 (SDA, SCL). Library handles nibble-mode communication and backlight control.

Specifications

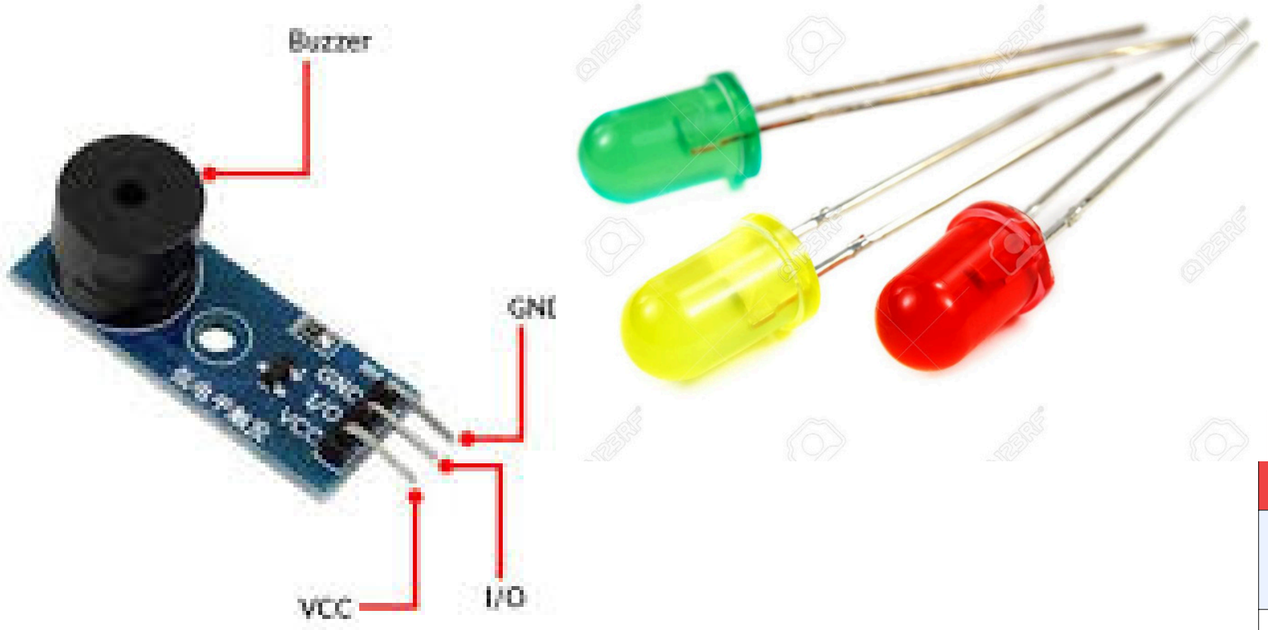
Parameter	Value
Interface	I2C via PCF8574
I2C Address	0x27 (default)
Supply Voltage	5V
Display Size	16 columns × 2 rows
SCL Pin	PB6 — I2C1_SCL
SDA Pin	PB7 — I2C1_SDA

Role in Project

Line 1: Temperature + Air Quality status. Line 2: Smoke level + Motion flag. Updated every second from main loop.

Part 3 · Member 3

Buzzer & LED Indicators



Working Principle

Active buzzer emits sound when 3.3V GPIO output is set HIGH. Three LEDs (Green/Yellow/Red) indicate air quality tier — each protected by a 220Ω current-limiting resistor to prevent GPIO pin damage.

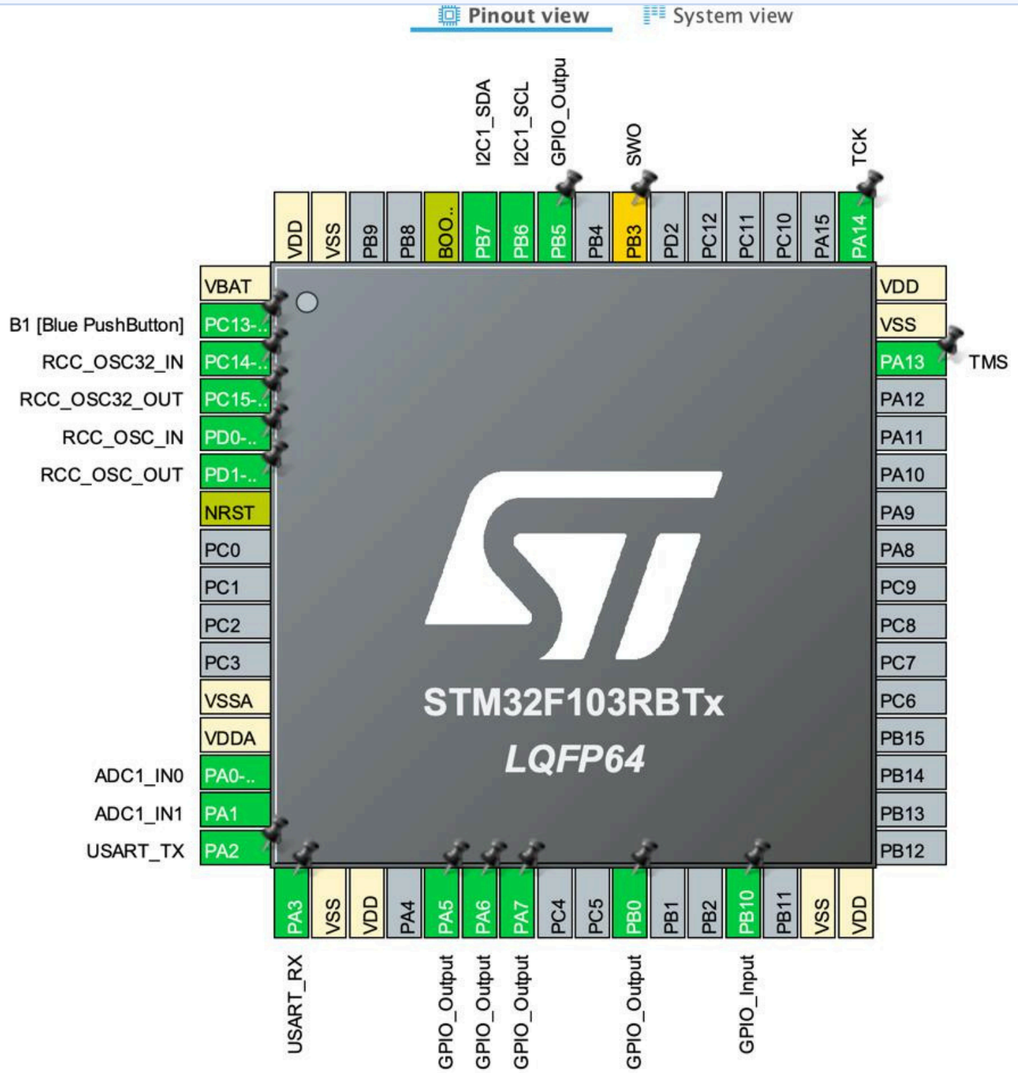
Specifications

Parameter	Value
Buzzer Type	Active (3.3–5V)
Buzzer Pin	PB0 — GPIO_OUTPUT
Green LED	PA5 — Air Quality Good (<2000)
Yellow LED	PA6 — Air Quality Poor (2000–2500)
Red LED	PA7 — Danger (>2500 or smoke)
Resistors	220Ω × 3 in series with each LED

Role in Project

Three-tier escalating alert: Green → Yellow → Red+Buzzer. Only one LED active at a time. Buzzer fires with Red only.

Circuit & Wiring Diagram



Pin Connection Reference

PA0	MQ-135 AOUT → ADC1_IN0
PA1	MQ-2 AOUT → ADC1_IN1
PA2	UART2 TX → USB to PC
PA3	UART2 RX
PA5	LED Green (220Ω series)
PA6	LED Yellow (220Ω series)
PA7	LED Red (220Ω series)
PB0	Buzzer
PB6	LCD SCL (I2C1)
PB7	LCD SDA (I2C1)
PB9	DS18B20 DATA + 4.7kΩ pull-up
PB10	PIR OUT → GPIO_INPUT

Sensor Data Flow Diagram

SENSORS	INTERFACES	STM32 F103RB	OUTPUTS
MQ-135 Air Quality	ADC1 PA0, PA1	Read ADC & Convert	LCD Update
MQ-2 Smoke/Gas	1-Wire PB9	1-Wire Poll & Parse	LED + Buzzer
DS18B20 Temperature	GPIO_IN PB10	Decision Logic	UART TX 115200
PIR Motion	I2C1 PB6, PB7	Format UART String	Python Graph

Code

```
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Smart Air Quality Monitor - Final Version with Tone Buzzer
  ******************************************************************************
  */
/* USER CODE END Header */

#include "main.h"
#include <stdio.h>
#include <string.h>

/* ── Defines ──────────────────────────────────────────────────────────────*/
#define LCD_ADDR (0x27 << 1)
#define DS18B20_PIN GPIO_PIN_8
#define DS18B20_PORT GPIOA
#define BUZZER_PIN GPIO_PIN_0
#define BUZZER_PORT GPIOB
#define MQ135_WARN 3200
#define MQ135_DANGER 3700
#define MQ2_SMOKE 4000

/* ── Handles ──────────────────────────────────────────────────────────────*/
ADC_HandleTypeDef hadc1;
I2C_HandleTypeDef hi2c1;
UART_HandleTypeDef huart2;

/* ── Global variables ─────────────────────────────────────────────────────*/
uint32_t mq135_val = 0;
uint32_t mq2_val = 0;
uint8_t pir_state = 0;
float ds_temp = 0.0f;
uint8_t ds_ok = 0;
uint8_t buzzer_on = 0;
char uart_buf[100];

/* ── Function prototypes ──────────────────────────────────────────────────*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_I2C1_Init(void);
static void MX_USART2_UART_Init(void);

static void DWT_Init(void);
static void delay_us(uint32_t us);
uint32_t Read_ADC(uint32_t channel);
void Set_LED(uint8_t red, uint8_t yellow, uint8_t green);
void Buzzer_Tone(uint32_t frequency, uint32_t duration_ms);
void Buzzer_Off(void);

static void DS18B20_SetOutput(void);
static void DS18B20_SetInput(void);
static uint8_t DS18B20_Reset(void);
static void DS18B20_WriteBit(uint8_t bit);
static uint8_t DS18B20_ReadBit(void);
```

Code

```
static void DS18B20_WriteByte(uint8_t byte);
static uint8_t DS18B20_ReadByte(void);
static void DS18B20_ReadTemp(void);

static void LCD_Write(uint8_t data);
static void LCD_Nibble(uint8_t nibble, uint8_t rs);
static void LCD_Cmd(uint8_t cmd);
static void LCD_Data(uint8_t data);
static void LCD_Init(void);
static void LCD_Cursor(uint8_t row, uint8_t col);
static void LCD_Print(const char *str);
static void LCD_Update(void);

/* ── DWT Delay ──────────────────────────────────────────────────────*/
static void DWT_Init(void) {
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us) {
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000UL);
    while ((DWT->CYCCNT - start) < ticks);
}
```

```
static void delay_us(uint32_t us) {
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000UL);
    while ((DWT->CYCCNT - start) < ticks);
}

/* ── ADC ────────────────────────────────────────────────────────────*/
uint32_t Read_ADC(uint32_t channel)
{
    ADC_ChannelConfTypeDef sConfig = {0};
    sConfig.Channel = channel;
    sConfig.Rank = ADC_REGULAR_RANK_1;
    sConfig.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);
    HAL_ADC_Start(&hadc1);
    HAL_ADC_PollForConversion(&hadc1, 10);
    uint32_t val = HAL_ADC_GetValue(&hadc1);
    HAL_ADC_Stop(&hadc1);
    return val;
}

/* ── LEDs ───────────────────────────────────────────────────────────*/
void Set_LED(uint8_t red, uint8_t yellow, uint8_t green)
{
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, green ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, yellow ? GPIO_PIN_SET : GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, red ? GPIO_PIN_SET : GPIO_PIN_RESET);
}
```

Code

```
/* ── Buzzer Tone (like Arduino tone()) ──────────────────────────────*/
void Buzzer_Tone(uint32_t frequency, uint32_t duration_ms)
{
  if (frequency == 0) {
    HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_SET);
    HAL_Delay(duration_ms);
    return;
  }
  uint32_t half_period_us = 1000000 / (frequency * 2);
  uint32_t cycles = (frequency * duration_ms) / 1000;

for (uint32_t i = 0; i < cycles; i++) {
    HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_RESET);
    delay_us(half_period_us);
    HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_SET);
    delay_us(half_period_us);
  }
}

void Buzzer_Off(void)
{
  HAL_GPIO_WritePin(BUZZER_PORT, BUZZER_PIN, GPIO_PIN_SET);
}
/* ── DS18B20 ────────────────────────────────────────────────────────*/
static void DS18B20_SetOutput(void)
{
  GPIO_InitTypeDef g = {0};
  g.Pin = DS18B20_PIN;
  g.Mode = GPIO_MODE_OUTPUT_PP;
  g.Pull = GPIO_NOPULL;
  g.Speed = GPIO_SPEED_FREQ_HIGH;
  HAL_GPIO_Init(DS18B20_PORT, &g);
}

static void DS18B20_SetInput(void)
{
  GPIO_InitTypeDef g = {0};
  g.Pin = DS18B20_PIN;
  g.Mode = GPIO_MODE_INPUT;
  g.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(DS18B20_PORT, &g);
}

static uint8_t DS18B20_Reset(void)
{
  DS18B20_SetOutput();
  HAL_GPIO_WritePin(DS18B20_PORT, DS18B20_PIN, GPIO_PIN_RESET);
  delay_us(480);
  DS18B20_SetInput();
  delay_us(70);
  uint8_t presence = !HAL_GPIO_ReadPin(DS18B20_PORT, DS18B20_PIN);
  delay_us(410);
  return presence;
}
```

Code

```
static void DS18B20_WriteBit(uint8_t bit)
{
    DS18B20_SetOutput();
    HAL_GPIO_WritePin(DS18B20_PORT, DS18B20_PIN, GPIO_PIN_RESET);
    delay_us(2);
    if (bit)
        HAL_GPIO_WritePin(DS18B20_PORT, DS18B20_PIN, GPIO_PIN_SET);
    delay_us(60);
    HAL_GPIO_WritePin(DS18B20_PORT, DS18B20_PIN, GPIO_PIN_SET);
    delay_us(2);
}

static uint8_t DS18B20_ReadBit(void)
{
    DS18B20_SetOutput();
    HAL_GPIO_WritePin(DS18B20_PORT, DS18B20_PIN, GPIO_PIN_RESET);
    delay_us(2);
    DS18B20_SetInput();
    delay_us(12);
    uint8_t bit = HAL_GPIO_ReadPin(DS18B20_PORT, DS18B20_PIN);
    delay_us(50);
    return bit;
}

static void DS18B20_WriteByte(uint8_t byte)
{
    }
}
```

```
if (!DS18B20_Reset()) { ds_ok = 0; return; }
    DS18B20_WriteByte(0xCC);
    DS18B20_WriteByte(0x44);
    HAL_Delay(750);

    if (!DS18B20_Reset()) { ds_ok = 0; return; }
    DS18B20_WriteByte(0xCC);
    DS18B20_WriteByte(0xBE);

    uint8_t byte0 = DS18B20_ReadByte();
    uint8_t byte1 = DS18B20_ReadByte();

    int16_t raw = (int16_t)((byte1 <<  | byte0);
    ds_temp = (float)raw / 16.0f;

    if (raw != 0x0550 && raw != (int16_t)0xFFFF) {
        ds_ok = 1;
    } else {
        ds_ok = 0;
    }
}

/* ── LCD ──────────────────────────────────────────────────────────────────*/
static void LCD_Write(uint8_t data)
{
    HAL_I2C_Master_Transmit(&hi2c1, LCD_ADDR, &data, 1, HAL_MAX_DELAY);
}
```

Code

```
static void LCD_Nibble(uint8_t nibble, uint8_t rs)
{
    uint8_t data = (nibble & 0xF0) | 0x08 | rs;
    LCD_Write(data | 0x04); HAL_Delay(1);
    LCD_Write(data & ~0x04); HAL_Delay(1);
}

static void LCD_Cmd(uint8_t cmd)
{
    LCD_Nibble(cmd & 0xF0, 0x00);
    LCD_Nibble((cmd << 4) & 0xF0, 0x00);
    HAL_Delay(2);
}

static void LCD_Data(uint8_t data)
{
    LCD_Nibble(data & 0xF0, 0x01);
    LCD_Nibble((data << 4) & 0xF0, 0x01);
    HAL_Delay(1);
}

static void LCD_Init(void)
{
    HAL_Delay(50);
    LCD_Nibble(0x30, 0); HAL_Delay(5);
    LCD_Nibble(0x30, 0); HAL_Delay(1);
    LCD_Nibble(0x30, 0); HAL_Delay(1);
    LCD_Nibble(0x20, 0); HAL_Delay(1);
    LCD_Cmd(0x28);
    LCD_Cmd(0x0C);
    LCD_Cmd(0x06);
    LCD_Cmd(0x01);
    HAL_Delay(2);
}

static void LCD_Cursor(uint8_t row, uint8_t col)
{
    LCD_Cmd(((row == 0) ? 0x80 : 0xC0) + col);
}

static void LCD_Print(const char *str)
{
    while (*str) LCD_Data((uint8_t)*str++);
}

static void LCD_Update(void)
{
    char row1[17] = {0};
    char row2[17] = {0};

    if (ds_ok) {
        int t_int = (int)ds_temp;
int t_dec = (int)((ds_temp - t_int) * 10);
        snprintf(row1, sizeof(row1), "Temp:%d.%dC ", t_int,
t_dec);
    } else {
        snprintf(row1, sizeof(row1), "Temp: --.-C ");
    }

    if (mq2_val > MQ2_SMOKE || mq135_val >
MQ135_DANGER)
        snprintf(row2, sizeof(row2), "Air:DANGER P:%d ",
pir_state);
    else if (mq135_val > MQ135_WARN)
        snprintf(row2, sizeof(row2), "Air:WARN P:%d ", pir_state);
    else
        snprintf(row2, sizeof(row2), "Air:GOOD P:%d ", pir_state);

    LCD_Cursor(0, 0); LCD_Print(row1);
    LCD_Cursor(1, 0); LCD_Print(row2);
}
* ── Main
──────────────────────────────────────────────────────
──────────────────*/
int main(void)
{
```

Code

```
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_I2C1_Init();
  MX_USART2_UART_Init();
  DWT_Init();

  Buzzer_Off();

  /* Startup LED flash */
  Set_LED(1, 1, 1);
  HAL_Delay(300);
  Set_LED(0, 0, 0);

  /* Startup beep */
  Buzzer_Tone(1000, 100);
  HAL_Delay(50);
  Buzzer_Tone(2000, 100);
  Buzzer_Off();

  /* LCD startup */
  LCD_Init();
  LCD_Cursor(0, 0); LCD_Print(" Air Quality Mon");
  LCD_Cursor(1, 0); LCD_Print(" STM32F103RB ");
  HAL_Delay(2000);
  LCD_Cmd(0x01);
  HAL_Delay(2);

  uint32_t last_temp_read = 0;
  uint32_t last_update = 0;

  while (1)
  {
    uint32_t now = HAL_GetTick();

    /* Read MQ sensors + PIR */
    mq135_val = Read_ADC(ADC_CHANNEL_0);
    mq2_val = Read_ADC(ADC_CHANNEL_1);
    pir_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_10);

    /* LED + Buzzer logic */
    if (mq2_val > MQ2_SMOKE || mq135_val > MQ135_DANGER) {
      Set_LED(1, 0, 0);
      buzzer_on = 1;
    } else if (mq135_val > MQ135_WARN) {
      Set_LED(0, 1, 0);
      buzzer_on = 2; // warning tone
    } else {
      Set_LED(0, 0, 1);
      buzzer_on = 0;
    /* Apply buzzer tones */
    if (buzzer_on == 1) {
      /* Danger: alternating high-low alarm */
      Buzzer_Tone(2500, 80);
      Buzzer_Tone(800, 80);
    } else if (buzzer_on == 2) {
      /* Warning: single soft beep */
      Buzzer_Tone(1200, 50);
      Buzzer_Off();
      HAL_Delay(500);
    } else {
      Buzzer_Off();
    }

    /* Read DS18B20 every 2 seconds */
    if (now - last_temp_read >= 2000) {
      last_temp_read = now;
      DS18B20_ReadTemp();
    }

    /* LCD + UART every 500ms */
    if (now - last_update >= 500) {
      last_update = now;
      LCD_Update();

}
```

Part 4 · Member 4

Code

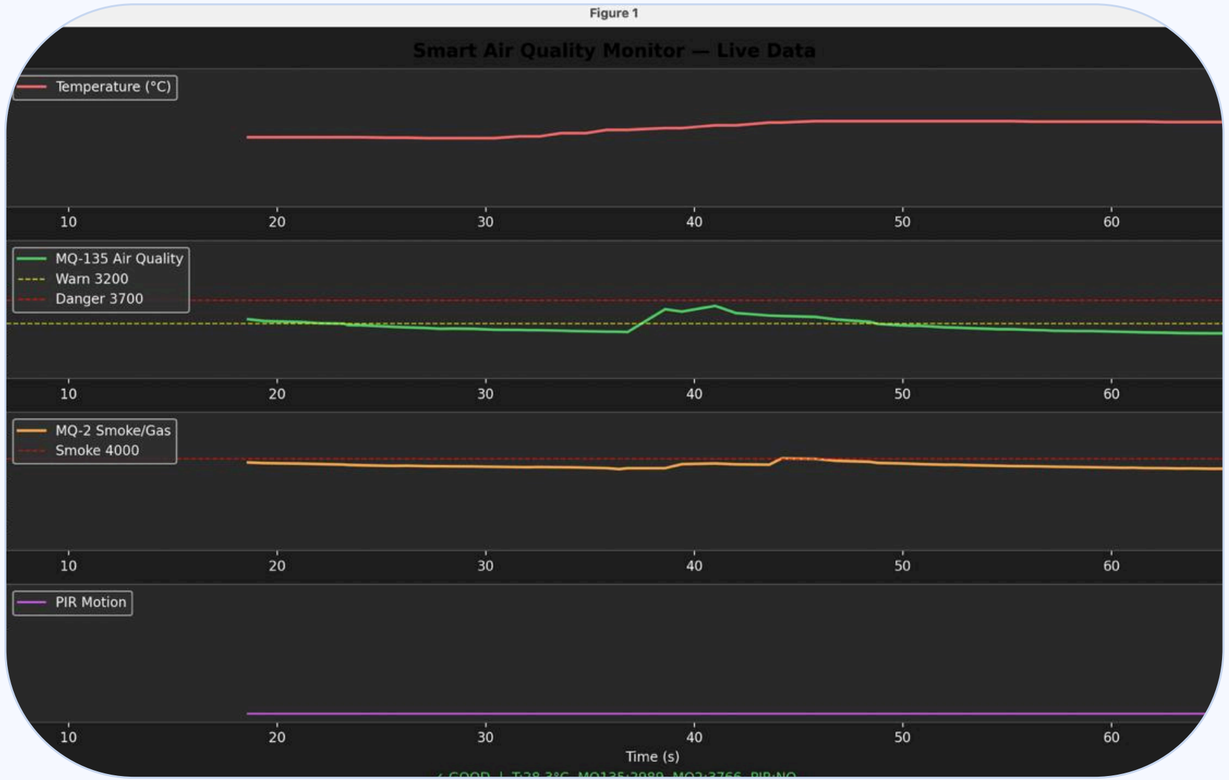
```
      int len = snprintf(uart_buf, sizeof(uart_buf),
                         "%d.%d,%u,%u,%d\r\n",
                         (int)ds_temp,
                         (int)((ds_temp - (int)ds_temp) * 10),
                         (unsigned int)mq135_val,
                         (unsigned int)mq2_val,
                         (int)pir_state);
       HAL_UART_Transmit(&huart2, (uint8_t*)uart_buf, len, 100);
     }

     HAL_Delay(10);
   }
}

/* ── Peripheral inits ──────────────────────────────────────────────────────*/
void SystemClock_Config(void)
{
   RCC_OscInitTypeDef RCC_OscInitStruct = {0};
   RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
   RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

   RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
   RCC_OscInitStruct.HSEState = RCC_HSE_ON;
   RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
   RCC_OscInitStruct.HSIState = RCC_HSI_ON;
   RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
```

Results & Testing



Test Observations

Normal Air Quality
ADC ~1200–3000 → Green LED ON

Poor Air Quality
ADC 3000–3700 → Yellow LED ON

Smoke / Danger
ADC > 3700 → Red LED + Buzzer

Temperature Read
DS18B20 returns 26–31°C in lab

Motion Detected
PIR HIGH → LCD shows 'MOTION!'

UART Format: 28.50,1350,980,0,1 → temp, MQ135, MQ2, smoke_flag, PIR_flag

CONCLUSION & ACKNOWLEDGEMENTS

What We Built

A fully integrated, real-time Smart Environmental Monitoring & Alert System on STM32 Nucleo-F103RB — detecting air quality, smoke, temperature and motion with live LCD display, LED/buzzer alerts, and a Python matplotlib dashboard over UART.

Key Takeaways

- ADC, 1-Wire, GPIO, I2C and UART on single MCU
- Three-tier escalating LED + buzzer alert logic
- Live Python pyserial + matplotlib dashboard
- Standalone, no internet, under ₹2,000 cost

Team Contributions

Swapnil Saraswat
24EC043
Problem, motivation, objectives, block diagram

Amisha Singh
24EC103
Use-cases, market data, comparison, future scope

Ananya Kasaudhan
24EC104
Sensor specs, wiring, data flow diagram

Pranjal Sharma
24EC111
Flowchart, code walkthrough, results

Thank You · Questions Welcome